

An honest read of GENOME on LoCoMo & TempBelief

GENOME ingests with zero LLM calls (over 8,000× cheaper at Haiku pricing), beats truncated full-context once history overflows the window, and keeps a deterministic, auditable bi-temporal record. It answers point-in-time queries better than Memo when sources state explicit dates (0.870 vs 0.676) — though we measured that edge disappearing on natural relative-dated language, and we say so. On saturated in-window QA it ties Memo. Where its features don't help, this report says that too.

WITHIN-HARNESS · IDENTICAL RESPONDER/JUDGE/EMBEDDER LoCoMo n=1,540 · TempBelief n=108 as-of

RESPONDER: CLAUDE HAIKU 4.5 PAIRED McNEMAR

SUMMARY

Six questions, six honest answers

Point-in-time ("as of time T") queries

Wins when the source states explicit dates (0.870 vs Memo 0.676, $p=0.0007$) but ties Memo on natural relative-dated language (0.759 vs 0.722, $p=0.82$). A conditional edge, not a universal one. (TempBelief.)

● WIN IF DATES EXPLICIT

Ingestion cost

o LLM calls per message vs Memo's 1.00; measured 8,433× cheaper, 9× faster.

● DECISIVE WIN

Overflow accuracy

On a 284k-token history, +0.409 accuracy at 80× less context than the stronger (recency-window) full-context baseline.

● DECISIVE WIN

In-window accuracy

0.851 vs Memo 0.855 vs full-context 0.863. No significant difference (McNemar $p>0.23$); GENOME's point estimate is nominally the lowest.

● PARITY

Synthesis (consolidation) IP

At equal token budget, summarize-vs-forget is a net wash: helps temporal, hurts single-hop.

● HONEST NULL

Hybrid / graph retrieval

Hybrid is measurably worse than dense; graph mode wasn't exercised here. Dense is the validated default.

● HONEST NULL

Why publish the nulls? A memory layer that only reports its wins can't be trusted on them. The two null results below are measured, reproducible, and make the two wins above credible.

The comparison is fair by construction

Every system in this report answers with the *same* model, is graded by the *same* judge (the Memo-verbatim judge prompt at temperature 0), embeds with the *same* encoder (`text-embedding-3-small`), and retrieves the *same* top-k. The only thing that varies is the memory layer. That isolates the layer as the independent variable and removes the usual confound where a "better" score just reflects a bigger answer model or a laxer judge.

Scores are reported over the 1,540 headline questions (LoCoMo categories 1–4: multi-hop, temporal, open-domain, single-hop). The adversarial category 5 is scored separately as an abstention test. Significance is paired McNemar on per-question correctness — the right test when the same questions are answered by both systems.

Two caveats we state up front. (1) The answer model and the judge are the same family (Haiku 4.5) — standard for LoCoMo harnesses, but a same-model-judge carries a self-preference risk that applies equally to every system here. (2) Numbers are single-run at a fixed seed; we report point estimates without confidence intervals, so treat sub-percent gaps as noise.

FINDING 06 • WIN (EXPLICIT-DATED SOURCES ONLY)

Point-in-time truth — but only when dates are explicit

The findings above are about *retrieval*. This one is a different primitive. GENOME maintains a contradiction-resolved, bi-temporal model of how facts change, recording each fact at its *domain-time* validity (when it became true in the world, parsed from the text) rather than wall-clock ingest time. So it can answer “what was true at time T?” even when facts are revealed out of chronological order. We tested this on TempBelief -- a deterministic, gold-validated benchmark where entities' attributes change over 6-8 sessions, ~1/3 of updates arrive out of order, and distractors must not move the belief. Same responder/judge/embedder; Memo run at its best (`infer=True` + `history()` traversal).

SPLIT	BELIEF-STATE	MEMO	DENSE	FULL-CONTEXT
as-of (point-in-time)	0.870	0.676	0.407	0.139
current-value	0.833	0.861	0.972	1.000
history	1.000	1.000	1.000	1.000
macro-J (3 splits)	0.901	0.846	0.793	0.713

Within-harness, 6 conversations, n=108 as-of. Source: `benchmarks/tempbelief_run.py` over `results/tempbelief/`.

0.870

BELIEF-STATE AS-OF VS MEMO
0.676

p = 0.0007

MCNEMAR VS MEMO (28 VS 7 OF
108)

0.97 / 1.00

KG EXTRACTION PRECISION /
DATE ACCURACY (AUDITED)

The honest read: on point-in-time queries, belief-state beats *every* baseline with significance (vs Memo $p=0.0007$, vs dense and full-context $p<10^{-5}$). Overwrite-based memory loses old values; raw retrieval and full-context return the *latest* value or can't disambiguate dates in a cluttered context. Only a bi-temporal store answers “what was true at T” reliably. This is **not judge leniency**: a full six-conversation audit finds the KG at 0.97 precision / 0.96 recall, and *every* captured fact is dated correctly (domain-time accuracy 1.000). **Honest tradeoff:** belief-state *loses* current-value (0.833 vs 0.97-1.0) -- its LLM extraction is noisier on trivial “latest value” lookups than reading the last mention.

The win does NOT survive natural language — we measured it. We re-ran the benchmark with every fact phrased relatively (“about two years ago”, “last year”) anchored to the message date, instead of a verbatim “Month Year”. The result: as-of belief 0.759 vs Memo 0.722, McNemar $p=0.82$ — a tie. The decisive verbatim result (0.870 vs 0.676) *collapsed*, because with relative dates Memo's LLM resolves them at query time about as well as GENOME resolves them at ingest. So this is a real advantage **only when the source states explicit, machine-parseable dates** (logs, structured notes, timestamped records) — NOT a general-conversation advantage. The durable belief-state value is the *deterministic, structured, auditable record* (below), not a QA-accuracy edge. Also: the history row is a partial-credit ceiling (all four at 1.000, uninformative); the real signal is as-of alone, not macro-J; 6 convs, one synthetic dataset, single seed.

FINDING 01 • PARITY

In-window accuracy is saturated

SYSTEM	HEADLINE ACC.	MULTI-HOP	TEMPORAL	OPEN-DOM.	SINGLE-HOP
full-context (upper bound)	0.863	0.883	0.748	0.531	0.938
Memo	0.855	0.890	0.863	0.479	0.882

SYSTEM	HEADLINE ACC.	MULTI-HOP	TEMPORAL	OPEN-DOM.	SINGLE-HOP
GENOME	0.851	0.851	0.798	0.500	0.911

Judge accuracy (J), n=1,540. GENOME = the parent-filtered variant; the plain dense variant is within a point (0.852). Source: `benchmarks/verdict.py` over `results/locomo_claude_v2/`.

Paired McNemar finds no significant difference between GENOME and either baseline: versus Memo, $\Delta=-0.004$ (p=0.747); versus full-context, $\Delta=-0.012$ (p=0.232). Both gaps are within noise — but note the direction: GENOME's point estimate is *nominally the lowest of the three*.

The honest read: LoCoMo in-window accuracy has a ceiling near 0.85 that every serious system hits. GENOME reaches it but sits a hair below both baselines; the differences aren't statistically significant, so the correct statement is “*no confirmed difference*,” not “*GENOME ahead*.” Either way, this is not where a memory layer earns its keep — and we don't claim an in-window accuracy win.

FINDING 02 • DECISIVE WIN

The win appears when history overflows

A memory layer exists for the case full-context can't handle: history larger than the window. We built a LoCoMo-Haystack of ~284k tokens (past a 200k window) and asked the same 120 questions of GENOME's constant-cost retrieval versus full-context truncated to several budgets — truncating from *both* ends (prefix = first-B tokens; recency = last-B tokens, what a real chat app keeps) so the baseline isn't a strawman.

0.817

GENOME @ 1,602 TOK/QUERY

0.408

BEST FULL-CONTEXT (RECENCY @128K)

+0.409

DELTA VS THE STRONGER BASELINE, 80× LESS CONTEXT

SYSTEM	ACCURACY	CONTEXT / QUERY
GENOME (retrieval)	0.817	1,602
full-context recency @128k	0.408	128,000
full-context prefix @128k	0.300	128,000

SYSTEM	ACCURACY	CONTEXT / QUERY
full-context recency @32k	0.125	32,000
full-context prefix @32k	0.133	32,000
full-context @8k (either end)	0.067–0.117	8,000

120 questions over a ~284k-token history. Recency (last-B) is the stronger full-context arm – it keeps later sessions. Source: benchmarks/haystack_report.py over results/haystack/ (+ haystack_recency.py).

The collapse holds across every category, against the stronger recency@128k baseline: GENOME leads single-hop 0.871 vs 0.500, multi-hop 0.810 vs 0.429, temporal 0.783 vs 0.217, open-domain 0.333 vs 0.000 (the last on n=6 – too small to weigh individually). The headline gap is not noise: paired McNemar of GENOME vs recency@128k gives $p \approx 8 \times 10^{-10}$ (55 questions GENOME alone got right, 6 the reverse). Neither truncation direction helps – with 284k of history no fixed window holds it all, so the answer model literally cannot see out-of-window evidence. Retrieval puts it back at 1/80th the context.

FINDING 03 • DECISIVE WIN

Ingestion costs almost nothing

GENOME stores a message by embedding it — no LLM in the write path. Memo runs an LLM fact-extraction on every message. We instrumented Memo's own LLM client to count its real calls and tokens over an identical 80-turn slice.

SYSTEM	LLM CALLS / MSG	TOKENS (80 TURNS)	WALL-CLOCK	COST (80 TURNS)
Memo	1.00	692,132	164.4 s	\$0.7119
GENOME (dense)	0	4,221	19.2 s	\$0.00008

Measured over one 80-turn run (fixed, no repeats). Mem0's LLM cost at Haiku pricing (\$1 / \$5 per Mtok in/out); GENOME's \$0.00008 is embedding pricing (0 LLM calls). The 8,433× multiplier is computed from the unrounded per-message costs, not the rounded cells shown. The 692k input tokens are Mem0's own extraction+update prompts, not injected by us. Source: benchmarks/ingest_cost.py (results/ingest_cost.log), Mem0's LLM client wrapped with a call/token counter.

8,433×

CHEAPER PER MESSAGE

~\$52 → \$0.006

FULL CORPUS, LINEARLY
EXTRAPOLATED

0

INGEST-TIME LLM CALLS

The honest read: the robust, model-independent fact is the call count — 0 vs 1.00 LLM call per message, which no pricing choice can change. The dollar multiplier does depend on the model priced: at Haiku pricing the full corpus is ~\$52 (Memo) vs ~\$0.006 (GENOME); a cheaper extraction model shrinks the ratio but it stays large, because GENOME's ingest LLM cost is structurally zero. One mechanism we did *not* measure but expect: Memo's per-message cost should rise as its store fills (each write ships existing memories to the LLM to decide an update), so linear extrapolation is if anything an underestimate. GENOME's differentiator is the zero in that first column.

FINDING 04 • HONEST NULL

Among modes we could fairly test, dense wins

GENOME ships dense, hybrid, and graph retrieval. Using LoCoMo's annotated evidence turns as ground truth, we measured how much gold evidence each mode surfaces — no LLM, so this isolates retrieval from the saturated answer model.

MODE	HIT@5	HIT@10	HIT@20	FULL-RECALL@10
dense	0.622	0.715	0.798	0.652
dense + recency rerank	0.586	0.708	0.798	0.645
hybrid	0.497	0.564	0.714	0.512

Mean fraction of gold evidence retrieved, 1,536 evidence-annotated questions. Source: `benchmarks/retrieval_quality.py`.

The honest read: among the modes this harness could fairly exercise, plain dense is best — hybrid is worse (0.564 vs 0.715 hit@10, a descriptive mean over 1,536 questions; keyword blending pulls in lexically-similar but wrong turns) and recency reranking is slightly worse (LoCoMo evidence spans all sessions). **Graph mode is omitted, not validated:** it needs an entity graph this offline store never

built, so it correctly fell back to dense and can't be scored here (we confirmed the store held 0 entities). We make *no* claim that graph helps — and an earlier full-pipeline graph variant scored far worse on answers (0.494), so the burden is on graph to prove itself. Dense is GENOME's validated default.

FINDING 05 · HONEST NULL

Synthesis is a tradeoff, not a free win

GENOME's signature idea is consolidation: rather than forget old memories under a budget, summarize them. We tested it the only fair way — equal token budget on both sides. Earlier internal runs showed a large synthesis "win," but that was a confound: a cluster-summary is a dense fact-list fed far more context than a pruned turn. Held to equal tokens, the picture is honest.

STRATEGY @ EQUAL BUDGET	ACCURACY	MULTI-HOP	TEMPORAL	SINGLE-HOP	OPEN-DOM.
PRUNE (forget old)	0.255	0.333	0.162	0.254	0.375
SYNTHESIZE (summarize old)	0.260	0.405	0.246	0.206	0.344

Storage budget S=3,000 tok, answer budget B=1,500 tok, n=584 (all 4 convs). Context fed was equal (1,473 vs 1,436 tok). Delta +0.005, McNemar p=0.861 — not significant. Source: benchmarks/budget_fair.py.

The honest read: aggregate delta is **+0.005 — statistically zero** (p=0.86). The per-category pattern is *directional, not individually significance-tested* (only the aggregate was): synthesis trends up on cross-session reasoning (temporal +0.085, multi-hop +0.072) and down on exact-fact lookup (single-hop -0.048, open-domain -0.031), consistent with the mechanism — a summary blurs the one verbatim detail a single-hop question needs. Read as a tendency, not proof. Consolidation is a **tunable tradeoff**: turn it on for temporal/analytical workloads, off for lookup-heavy ones. It is not a blanket accuracy win, and we don't sell it as one.

GUIDANCE

When GENOME is the right choice

Choose GENOME when ingestion cost matters (high message volume, or you don't want an LLM in your write path), or when conversation history exceeds the model's context window — the two cases this report measures a real advantage in.

Don't expect a higher in-window LoCoMo score than Memo — that metric is saturated and the two tie. If your entire history already fits the window and ingestion cost is irrelevant, full-context is a fine and simpler baseline;

a memory layer only starts paying off past that point.

REPRODUCIBILITY

Every number regenerates

```
python benchmarks/verdict.py           # Finding 01: in-window accuracy + McNemar
python benchmarks/haystack_report.py    # Finding 02: overflow / haystack
python benchmarks/ingest_cost.py --n 80 # Finding 03: measured ingestion cost
python benchmarks/retrieval_quality.py  # Finding 04: retrieval-mode hit-rate
python benchmarks/budget_fair.py        # Finding 05: token-fair synthesis ablation
```

All figures in this report trace to benchmarks/VERIFIED-NUMBERS.md, which pins each number to its source file. Within-harness config, temperature 0, Mem0-verbatim judge.

GENOME · LoCoMo benchmark report · generated from executed runs, no figures estimated except where labeled.
Methodology: within-harness comparison, identical responder (Claude Haiku 4.5) / judge / embedder / top-k across all systems; paired McNemar for significance.
Honest-benchmark principle: wins and nulls reported with equal prominence.